Walchand College of Engineering, Sangli

(Government Aided Autonomous Institute) · Department of Computer Science and Engineering

TY CSE · Mini Project-I (7CS345) · LA-2 Evaluation (Release 2), 2026-27

PromptOpt

Token-efficient prompt optimization for LLMs: rewrite vague prompts, explain every change, and measure that it helps

Team

Piush U. Gogi · 255200005
Nirzara A. Manade · 255200006
Siddhi S. Bolaikar · 255200008

Guide

Prof. A. S. Pawar

15 October 2026

Release 2 at a glance

-39.2%
total tokens, 482 test prompts

95.0%
state an output format (was 3.1%)

798
pytest tests passing + 13/13 UI tests

42/50
correct vs 37/50 vague (correctness suite)

What this review covers

1 Core modules: Stage A, B, C, IR + renderers, image mode, attachments, Compare, web app, database

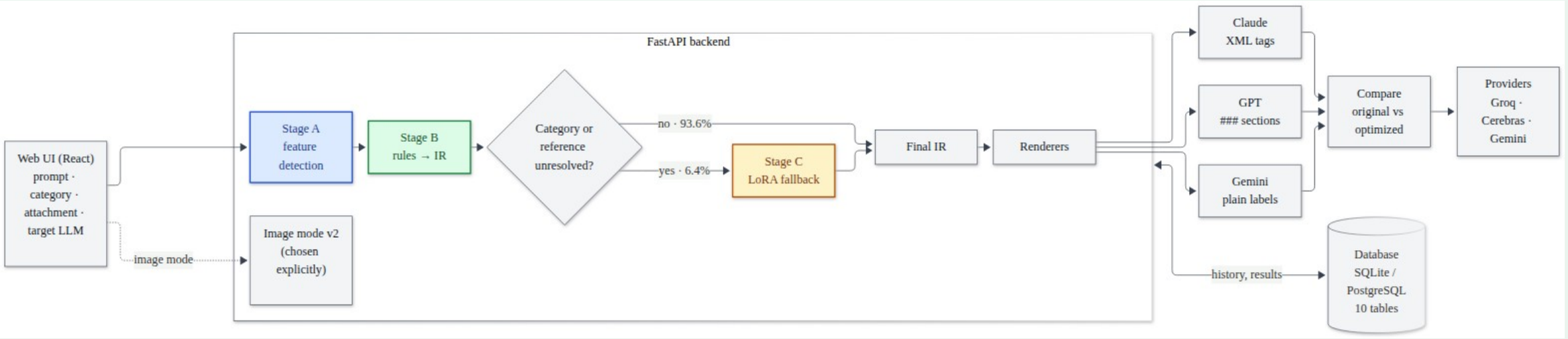
2 Transaction flow: submit → optimize → store (one commit) → Compare → History → retention

3 Unit testing · 4 Integration & system testing · 5 Working prototype

6 Test cases with real results, including honest failures

7 Git / GitHub · 8 Challenges and fixes · 9 Plans · Live demo

System architecture



Stage A

spaCy + regex + sentence-embedding classifier: category, missing format/constraints, filler, ambiguous refs

Stage B

15 deterministic rules B01-B15, every change logged; category rules only at confidence ≥ 0.6

Stage C

Qwen2.5-0.5B + LoRA, called only for unresolved fields; validated, else Stage B kept

Output

IR rendered per target LLM with token counts; Compare, History, 30-day retention

Stage A detects, Stage B fixes with 15 rules

Stage A · feature detection

- Task category + confidence (embedding classifier, keyword fallback)
- A02 missing output format
- A03 missing length / tone / audience / language
- A04 filler and repetition
- A05 ambiguous reference ("summarize this")

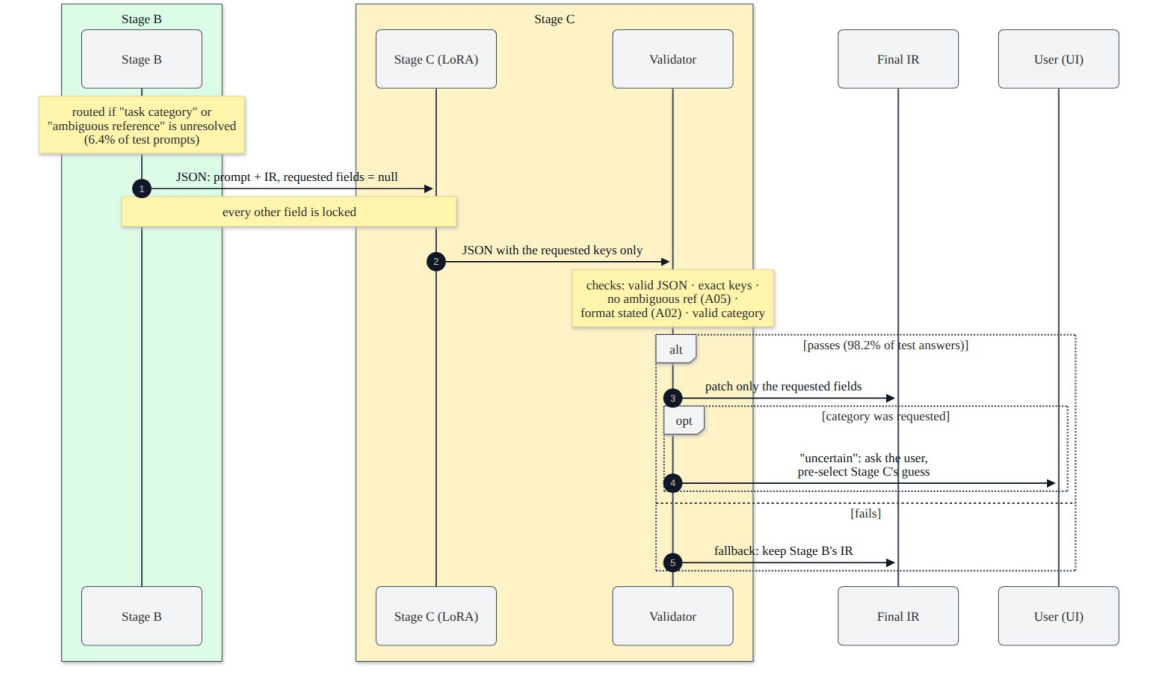
74.7% category accuracy on 482 test prompts (coding F1 0.98, classification 0.94)

Stage B · rules (each a unit-tested function)

B01 Remove filler	**B02** Remove duplicates	**B03** Output format
B04 Length limit	**B05** Programming language	**B06** Label set
B07 Structure / data block	**B08** Group fallback	**B09-B15** Attachment modifiers

3.1% → 95.0% of test prompts state an output format after Stage B; 6.4% (31 of 482) are routed to Stage C

Stage C: a small LoRA model, only where rules stop



5% → 98%

answers pass validation: base vs LoRA

22.6 → 90.3%

format stated on routed prompts

0.74 s

median per call, RTX 3050 (CPU 4.55 s)

6.4%

of test prompts routed to Stage C

Forced on every prompt, Stage C lowers task intent (0.886 → 0.769), so it stays a fallback. Its category is only a suggestion: the UI asks the user.

IR, renderers, attachments and image mode

Claude

```
<task>Write code to get
all
permutations of a
string.</task>
<constraints>- Use
Python.
</constraints>
<output_format>Return
only
the code
...</output_format>
```

GPT

```
### Task
Write code to get all
permutations of a string.
### Constraints
- Use Python.
### Output format
Return only the code ...
```

Gemini

```
Task: Write code to get
all
permutations of a string.
Constraints: Use Python.
Output format: Return
only
the code ...
```

Same content, layout per model · every rendering parsed back in the tests

Attachments B09-B15: image, PDF, PPTX, DOCX, spreadsheet, code, other; fire only with an attachment · **30/30** hand-made prompts correct

Image mode (separate, user-selected)

Held-out CLIP vs the user's prompt (30 prompts)

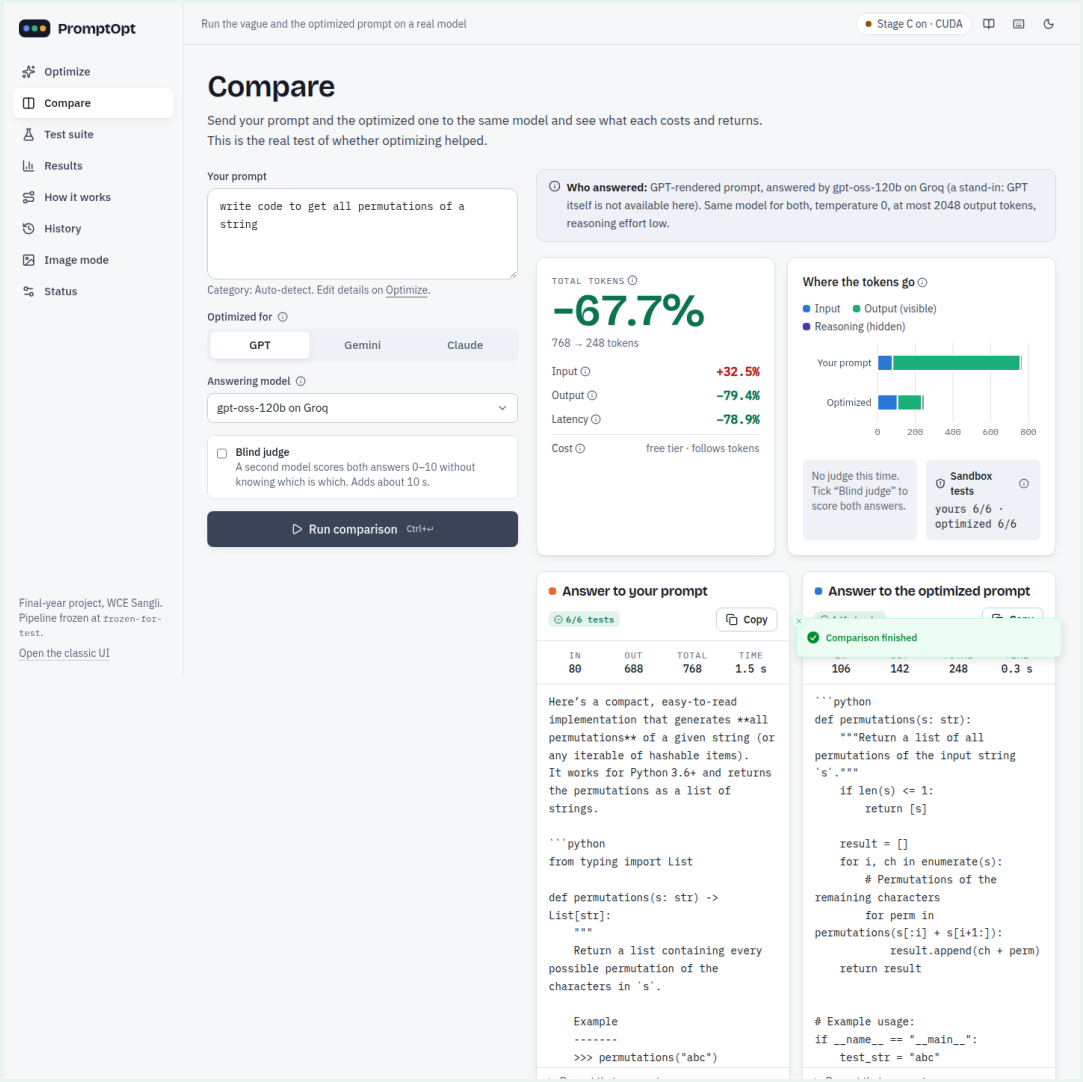
original
33.08

v1 defaults
30.78 (worse, p=0.043)

v2 suggestions
32.77 (no sig. harm)

Style kept in 12/12 styled prompts with v2

Web app and Compare on a real model



-67.7%

total tokens: 768 → 248 (live, gpt-oss-120b)

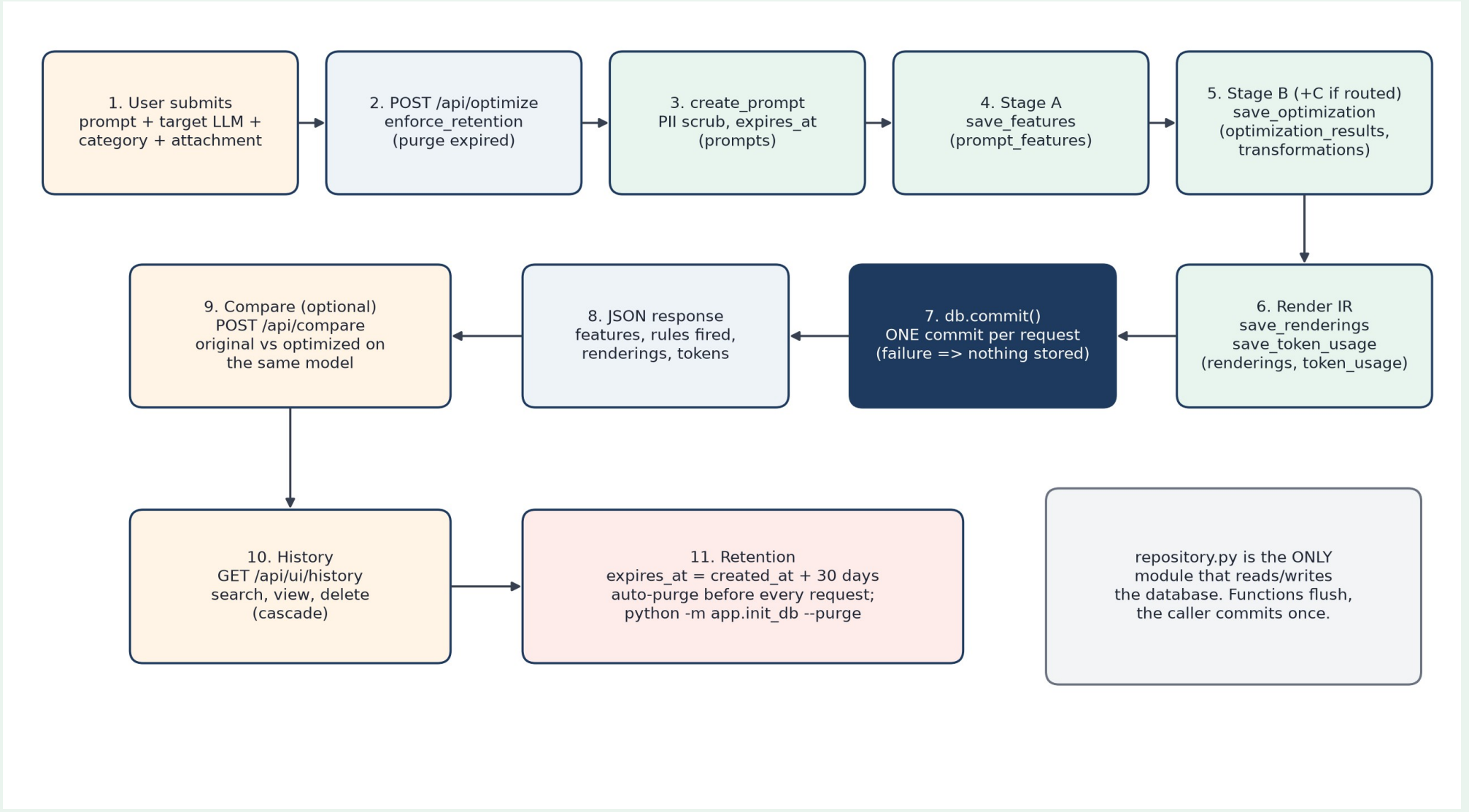
6/6 = 6/6

sandbox tests pass for both answers

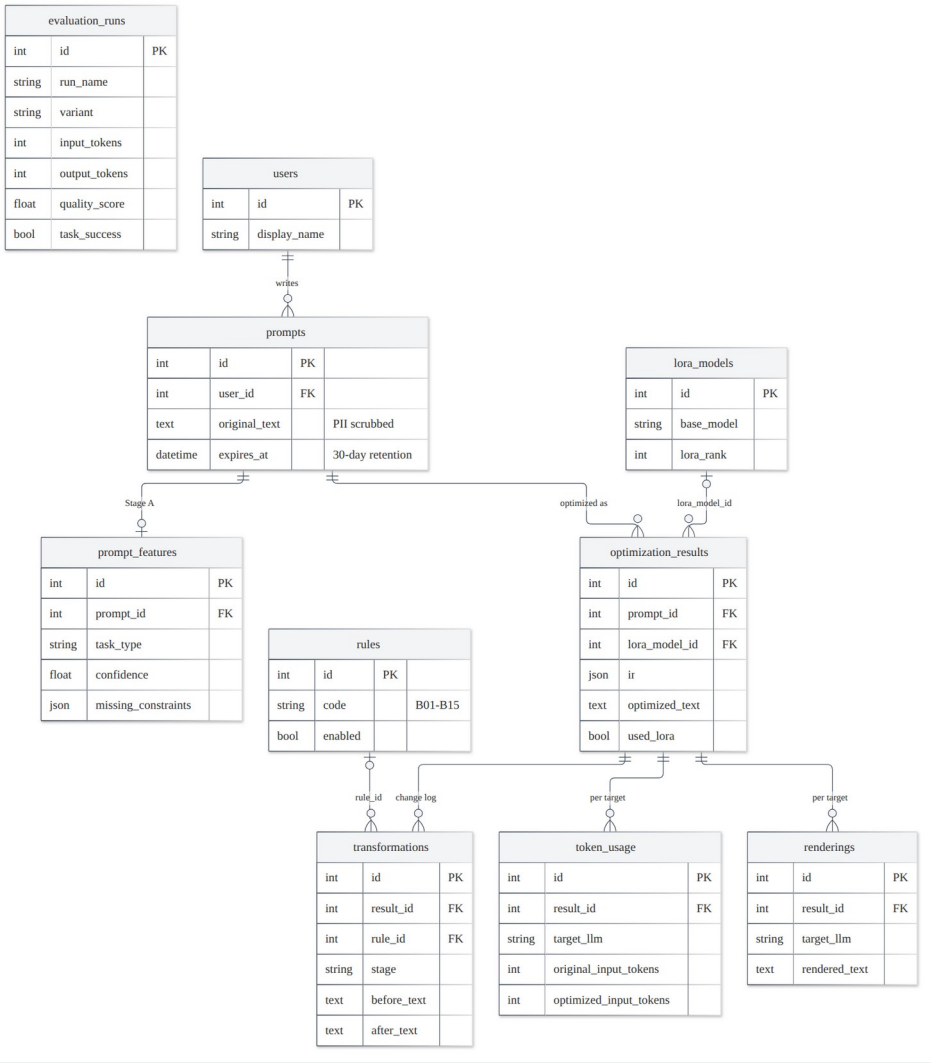
What the app does

- Optimize: every stage shown, rule by rule, with before/after text
- Compare: same model, temperature 0, tokens, latency, tests, optional blind judge
- History: search, view, delete; prompts expire after 30 days
- Status: providers' 24-hour usage, Stage C on CUDA, sandbox

Transaction flow: one request, one commit



Database layer: 10 tables, repository only

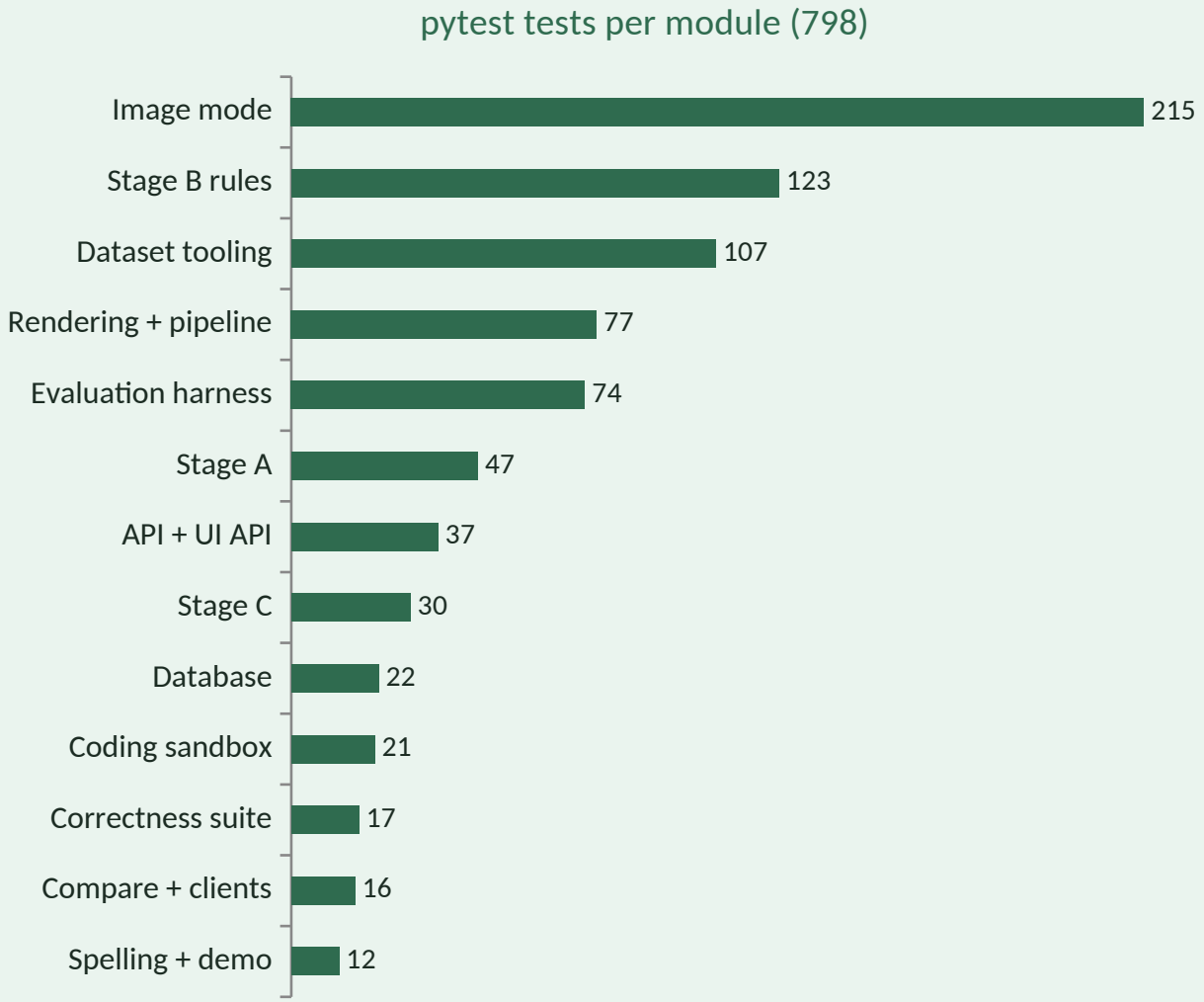


Written in one optimize transaction

Table	Repository call	Stores
prompts	create_prompt	PII-scrubbed text, expires_at
prompt_features	save_features	category, confidence, gaps
optimization_results	save_optimization	IR, optimized text, used_lora
transformations	save_optimization	one row per change, rule_id
renderings	save_renderings	prompt per target LLM
token_usage	save_token_usage	tokens per target

Cascade on delete · PII scrubbed before storing · 30-day retention enforced before every request

Unit testing: 798 tests, all passing



Run

```
cd backend
source .venv/bin/activate
python -m pytest -q
→ 798 passed in 24.6 s
```

Real examples

- `test_b05_defaults_to_python`
- `test_pii_is_scrubbed_before_storage`
- `test_retention_purge_cascades`
- `test_blank_prompt_is_rejected_with_422_not_500`
- `test_invalid_answers_fall_back_to_stage_b`

Integration and system testing

Freeze check

BYTE-IDENTICAL

482 test prompts, Stage A+B vs tag frozen-for-test (sha256 35f7d4dc…)

UI end-to-end

13 / 13

Playwright + Chrome: optimize, compare, suite, history, image mode, no console errors

API end-to-end

pytest

FastAPI TestClient: Stage C on/off, 422 errors, retention, PII-scrubbed Compare

Full pipeline

482 + 44

test split run once (2 Oct) · benchmark with real LLM + blind judge

Coding sandbox

20.7 → 34.5%

pass@1 strict, 29 Python items; bubblewrap: no network, read-only

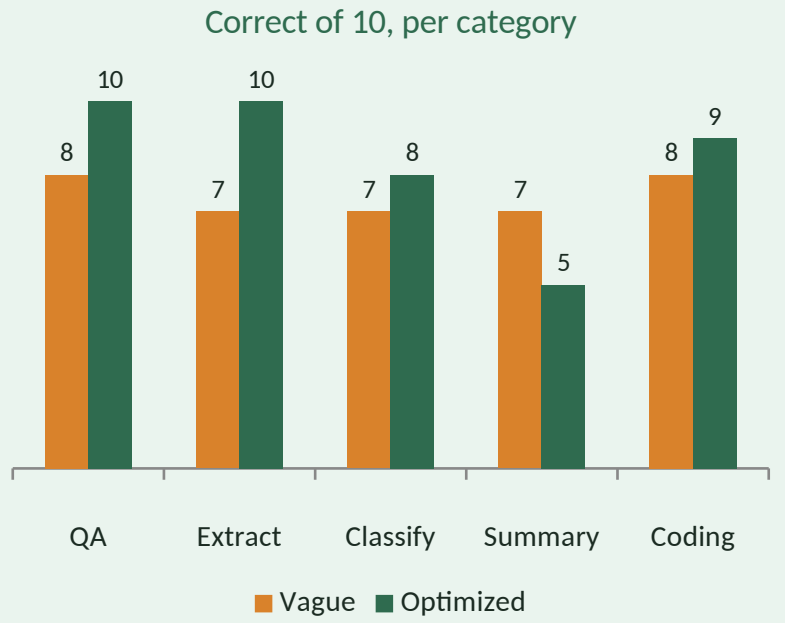
Correctness suite

37 → 42 / 50

hand-written cases with verifiable answers, gold human-reviewed

Test cases and correctness, including failures

ID	Case	Input	Result	Status
TC04	B01 removes filler	"hey can you please summarize the text for me?"	"Summarize the text."	PASS
TC16	PII scrubbed	mail rahul.k@example.com … 98765 43210	1 [EMAIL], 2 [PHONE]	PASS
TC26	Freeze check	482 test prompts	byte-identical	PASS
TC31	cqa-01 hostel fee	"if i pay on 18 july how much total??"	80300 (vague said 85300)	PASS
TC34	cls-09 attendance	"mark each student safe warning or detained"	got a Python script	FAIL
TC37	cod-10 recipe scaler	"write the scale fn for recipe app"	4/8 asserts	FAIL



Fixed 9 · broke 4 · McNemar p = 0.267 (not significant at n = 50)

Failures: Stage A misread a table as code (cls-09); two summaries 2-3 words over the limit; a Stage C coding case 4/8 asserts.

42/50

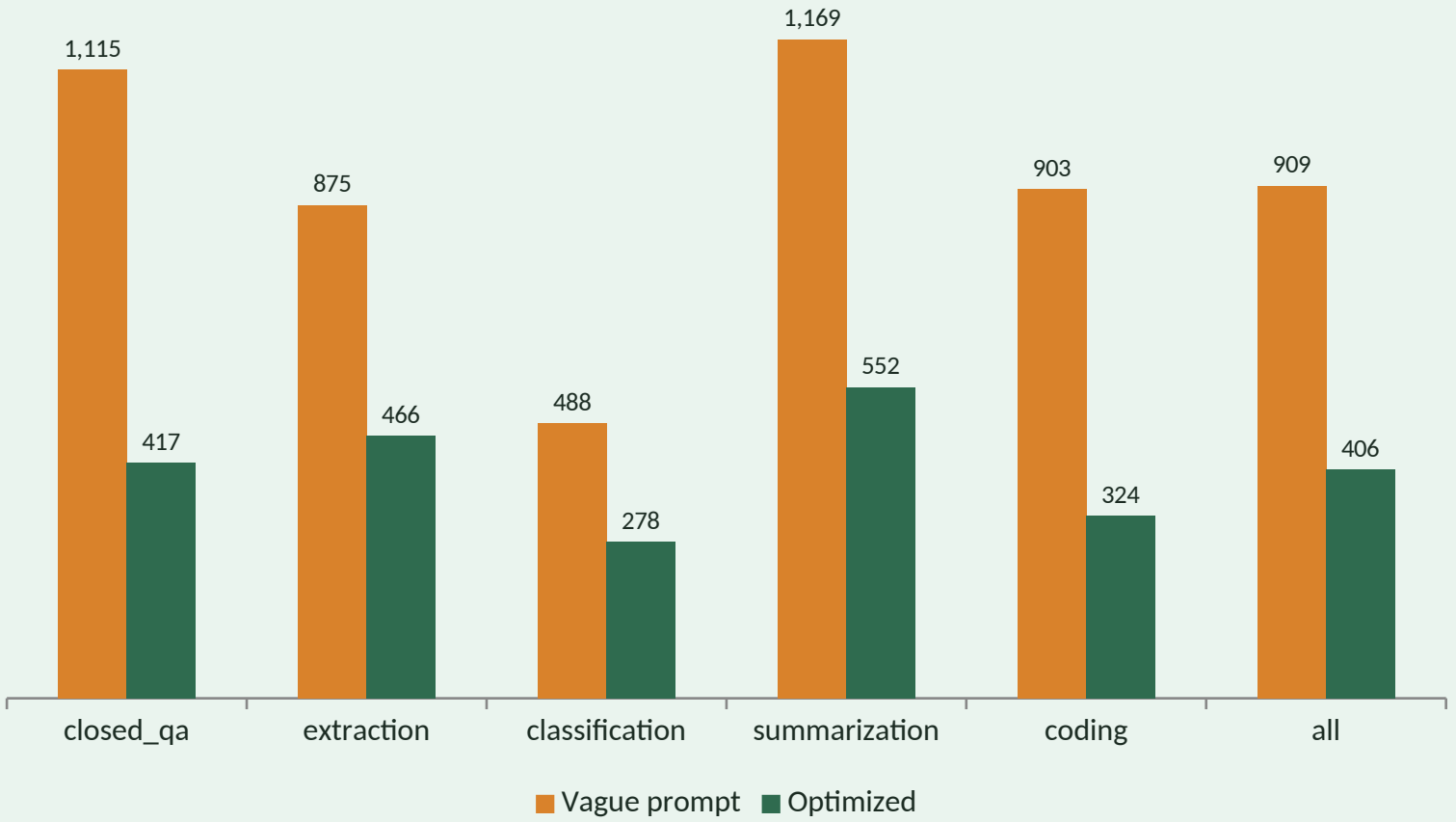
optimized (vague 37)

-29.6%

total tokens

Results: -39.2% total tokens on 482 test prompts

Mean total tokens per prompt (input + output), gpt-oss-120b



682 → 165

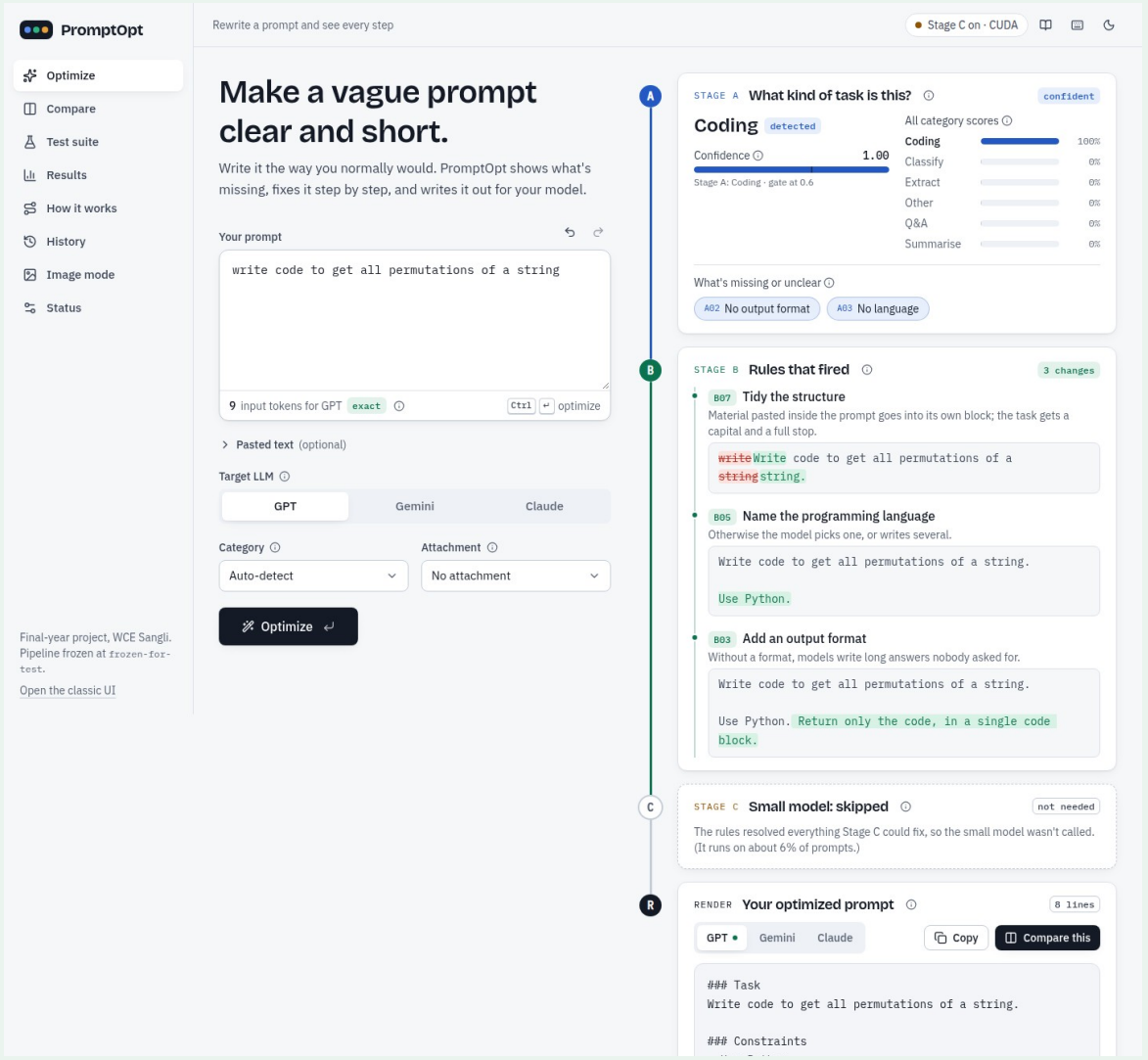
output tokens per prompt (input 226 → 241)

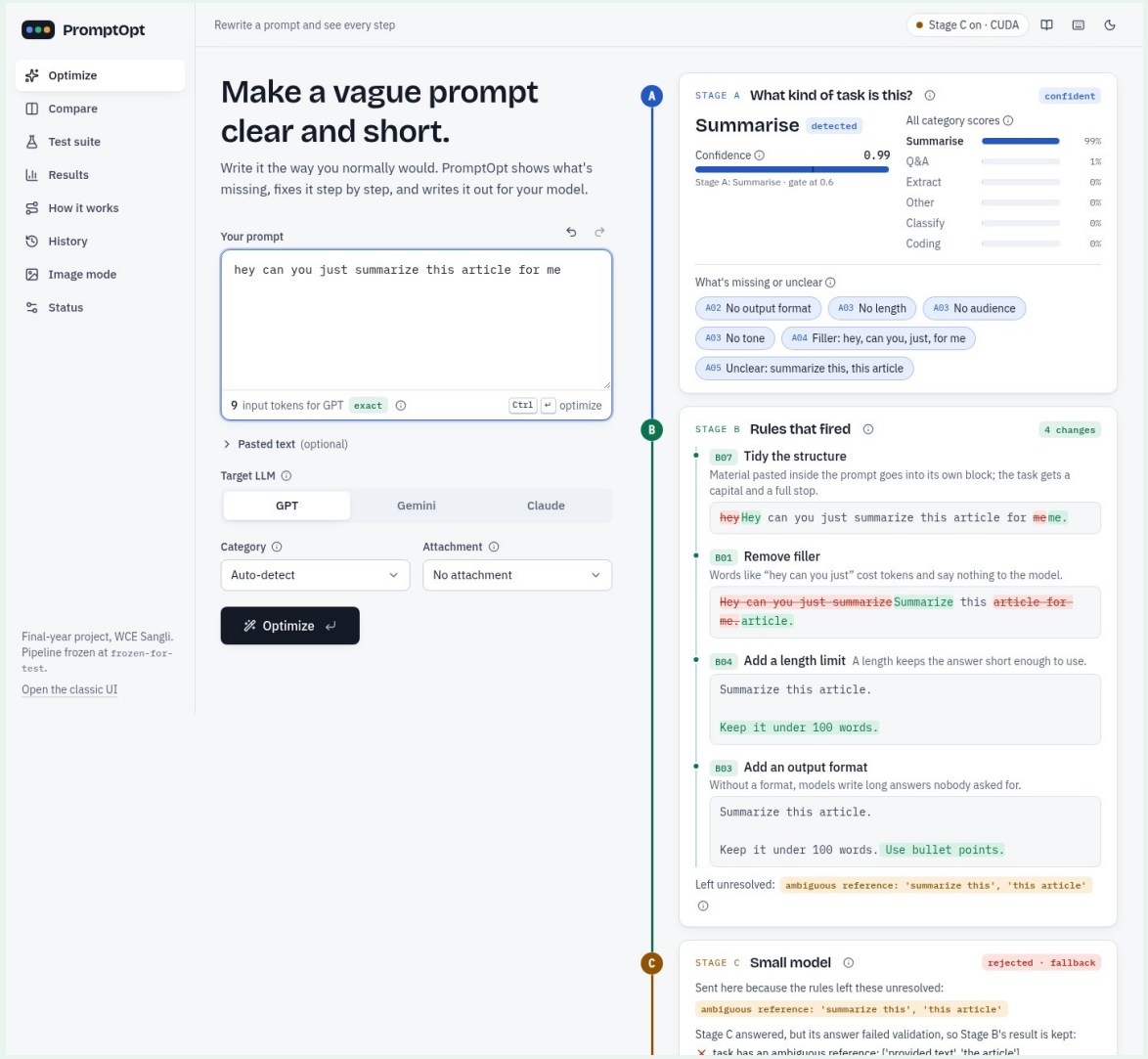
8.2 → 9.0

benchmark quality; task success 67% → 85%

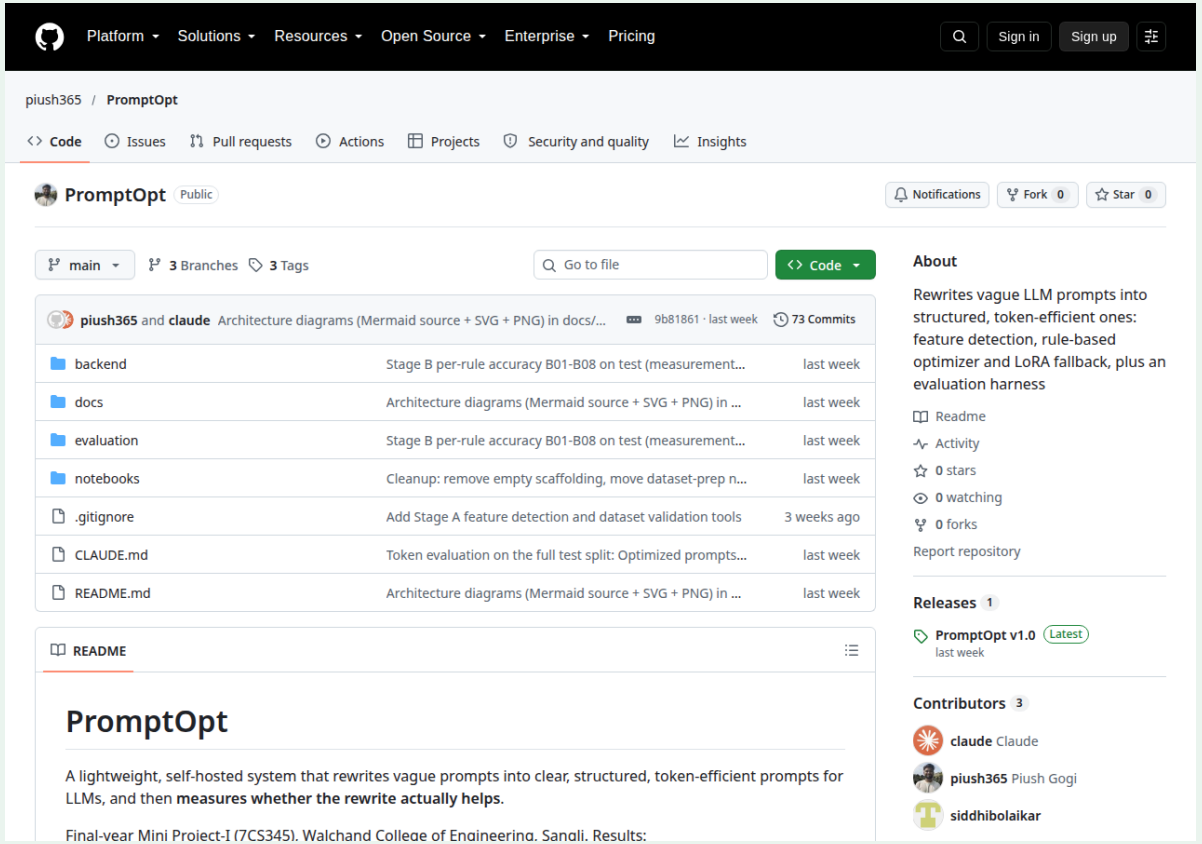
Limitation: 87 of 482 prompts cost more (short answers, mostly extraction)

Working prototype





Git and GitHub



github.com/piush365/PromptOpt

92	3	4
commits	branches	tags · 1 release

- frozen-for-test (27 Sep): code for the final numbers
- final-for-test (2 Oct): test split run once
- v1.0 (8 Oct): release + Stage C adapter
- Commits/month: Aug 7 · Sep 38 · Oct 47
- Tracking: roadmap + phase tables in CLAUDE.md, tags, release (no Issues/Projects used)

Challenges and how we solved them

Dataset generation bugs

LLM targets dropped data → deterministic repair: 349 prompts fixed, all logged

Kappa paradox

90% agreement, kappa 0.065 → report Gwet's AC1 0.92-0.96 too

API quota limits

Groq 200k tokens/day → 24-h usage ledger, 70% budget rule, resumable runs

Stage C hallucination

base model 5% valid → LoRA 98.2%, strict JSON contract, fallback to Stage B

Image mode v1 failure

defaults turned watercolor into photo → v2: suggestions only

Label / regex edge cases

"thank you so much" in a button → a unit test per edge case, B16 in the app

4 GB GPU training

micro-batch 1 × 16, bf16, checkpointing: 51 min; models loaded once

Rule: report it, don't hide it

After the freeze, bugs were reported, not fixed; failures are in the results.

Plans for the upcoming phases

1. **Lean mode**
   add only the format line when the answer is short
2. **Smarter routing**
   send label-set and typo prompts to Stage C
3. **Spelling normalisation**
   user-confirmed fixes before optimizing
4. **Blind user tests**
   prompts written outside the team
5. **Real GPT / Claude / Gemini**
   Compare with paid API keys
6. **Larger correctness suite**
   more cases and providers for significance
7. **Deployment**
   PostgreSQL, HTTPS, login, beyond the LAN

Live demo

1. Optimize: "write code to get all permutations of a string"
2. Stage C: "hey can you just summarize this article for me"
3. Compare on gpt-oss-120b: tokens, latency, tests
4. Image mode: "oil painting of a sailboat in a storm"
5. History, Test suite and Results pages

Thank you · questions welcome